ToddySoft Connect Modbus Driver - Ignition 8.1

Ignition 8.1 module for connecting to **Modbus TCP** devices — PLCs, meters, drives, I/O blocks and gateways. Coils, discrete inputs, input registers and holding registers are addressed directly, with a type suffix that tells the driver how to decode the bytes.

> **Modbus has no tag list and no device discovery, and this is a protocol fact rather than a gap in the driver.** Nothing on the wire enumerates addresses or announces devices, so you write addresses by hand from the device's Modbus map. Every other ToddySoft Connect module browses a controller for its symbols; this one cannot, and no setting will change that.

Installation

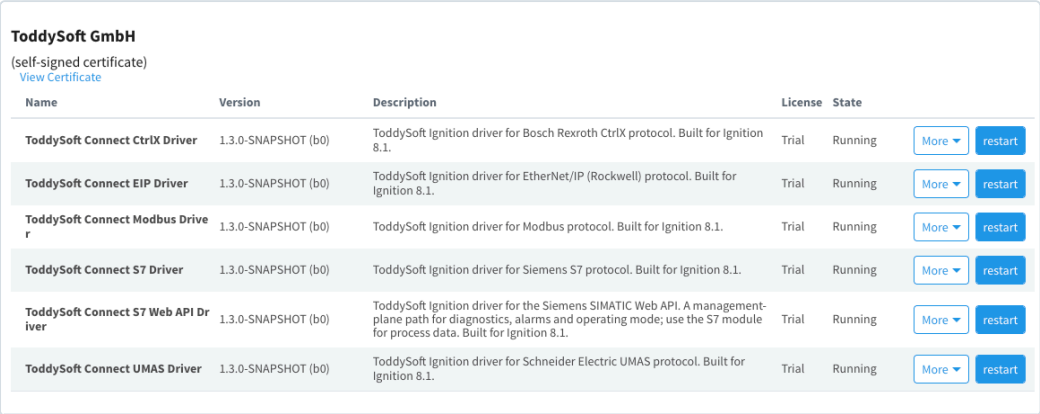
> **This module is for Ignition 8.1 only.** It is built against the Ignition 8.1 SDK and will not run on any other major version. For Ignition 8.3, install the separate `…-8.3.modl` build instead — the two are not interchangeable, and each gateway needs the build that matches it.
>
> **Ignition 8.0 and earlier are not supported and cannot be.** The module is built on the device-driver API that Ignition introduced with 8.1, so on 8.0 the gateway fails to load it with `NoClassDefFoundError: … opcua.server.api.AbstractDeviceModuleHook`.
>
> Both builds carry the same name in the gateway's module list, so tell them apart by the file you installed or by the module description, which names the Ignition version.
>
> **Ignition's OPC UA module must be installed and running.** This is an OPC UA device driver: it registers with Ignition's own OPC UA server, and every tag it exposes is served through it. With that module absent the gateway cannot load this one, and the log fills with `ClassNotFoundException` / `NoClassDefFoundError` naming `com.inductiveautomation.opcua…` classes. The OPC UA module ships with Ignition and is installed by default, so in practice this bites only on a gateway where it has been removed or disabled. **Ignition 8.1 cannot refuse the module for you** — its descriptor has no way to mark a dependency as required, a mechanism that arrived in 8.3 — but it does name the cause plainly, one line before the stack trace. In `logs/wrapper.log`, look for:
>
> `Module "com.toddysoft.connect.…" depends on "com.inductiveautomation.opcua", but it doesn't exist.`
>
> It then attempts to load the module anyway, which produces the class errors above. Ignition's own drivers behave identically in this situation, so this is how the platform works and not a fault in this module.

The ToddySoft Connect modules are signed with a self-signed certificate (issued to *ToddySoft GmbH*) rather than a publicly accredited code-signing certificate. Ignition therefore asks you to review and accept that certificate once during installation - it is then remembered for all future ToddySoft modules signed with the same certificate.

1. Open the Ignition Gateway web interface and navigate to **Config > System > Modules**. Scroll to the bottom and click **Install or Upgrade a Module...**.
2. Choose the `ToddySoft-Connect-Modbus-Driver-8.1.modl` file and click **Install**.

3. Ignition shows the self-signed certificate details (subject and issuer *ToddySoft GmbH*) together with a warning that the module was not written by Inductive Automation. Review the certificate, tick **I understand the risk and want to install this module** and click **Add Certificate and Install Module**.

4. The module is now trusted and listed under the **ToddySoft GmbH (self-signed certificate)** group with the state **Running**.



Before you start: what you need to know about the device

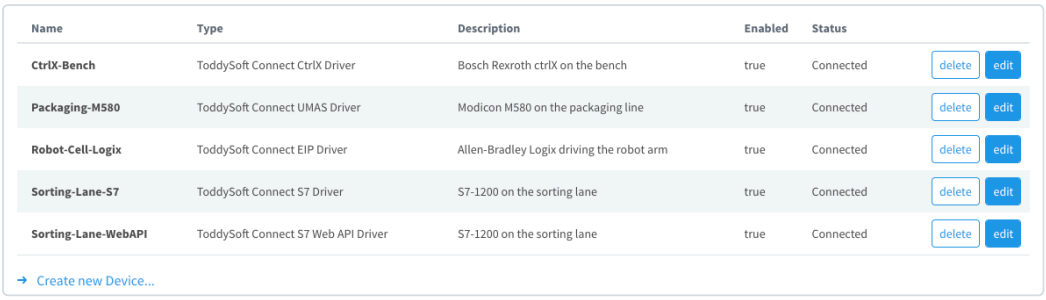
- **Its Modbus map.** Which registers hold what, and in which representation. There is no way to ask the device, so this comes from its documentation.
- **Its unit identifier** (slave id), if it is not `1`.
- **Its byte order** for anything wider than one register. Modbus defines the order of *registers* but not of bytes within a multi-register value, so vendors differ. See *Default Byte Order*.
- **TCP port 502** reachable from the gateway host, unless the device listens elsewhere.

There are no credentials: Modbus is unauthenticated. Anything on the network that can reach the port can read and write it, which is worth knowing when deciding where the gateway sits.

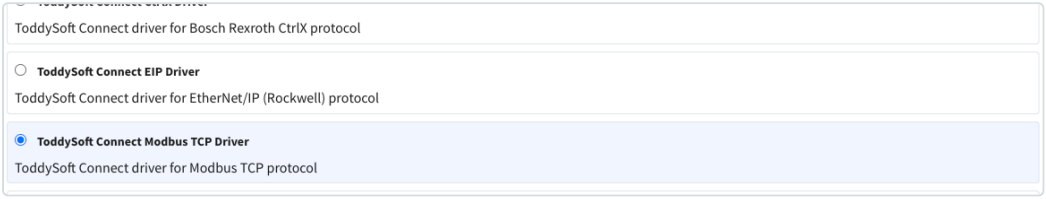
> **This module speaks Modbus TCP only**, and the device type says so — it is listed as *ToddySoft Connect Modbus TCP Driver*, while the module itself appears as *ToddySoft Connect Modbus Driver* in the modules list. The underlying driver also implements Modbus RTU and ASCII over serial, UDP and TLS, but the Ignition module emits a `modbus-tcp://` connection and exposes no transport selector, so those are not reachable from here. A serial device needs a Modbus gateway in front of it.

Creating a Device Connection

1. Go to **Config > OPC UA > Device Connections**.



2. Click **Create new Device...**, select **ToddySoft Connect Modbus TCP Driver** as the device type and click **Next**.



3. Configure the connection settings (see the sections below) and click **Save**.

4. The new device then appears in the Device Connections list.



> **There is no device discovery for this driver on Ignition 8.1**, and no *Test Connection* action. Modbus devices announce nothing; the Ignition 8.3 module finds them by probing addresses, which the 8.1 configuration page has no place for.

Setting blocks

The configuration is organized into categories: **General**, **Transport Settings**, **Modbus Settings**, **TCP Advanced Settings** and **Audit Log Settings**.

General

Assigns a name (and optional description) to the connection and controls whether it is enabled.



Setting	Description	Default
Name	Unique name for this device connection	
Description	Optional description for this connection	
Enabled	Whether the connection is active	`true`

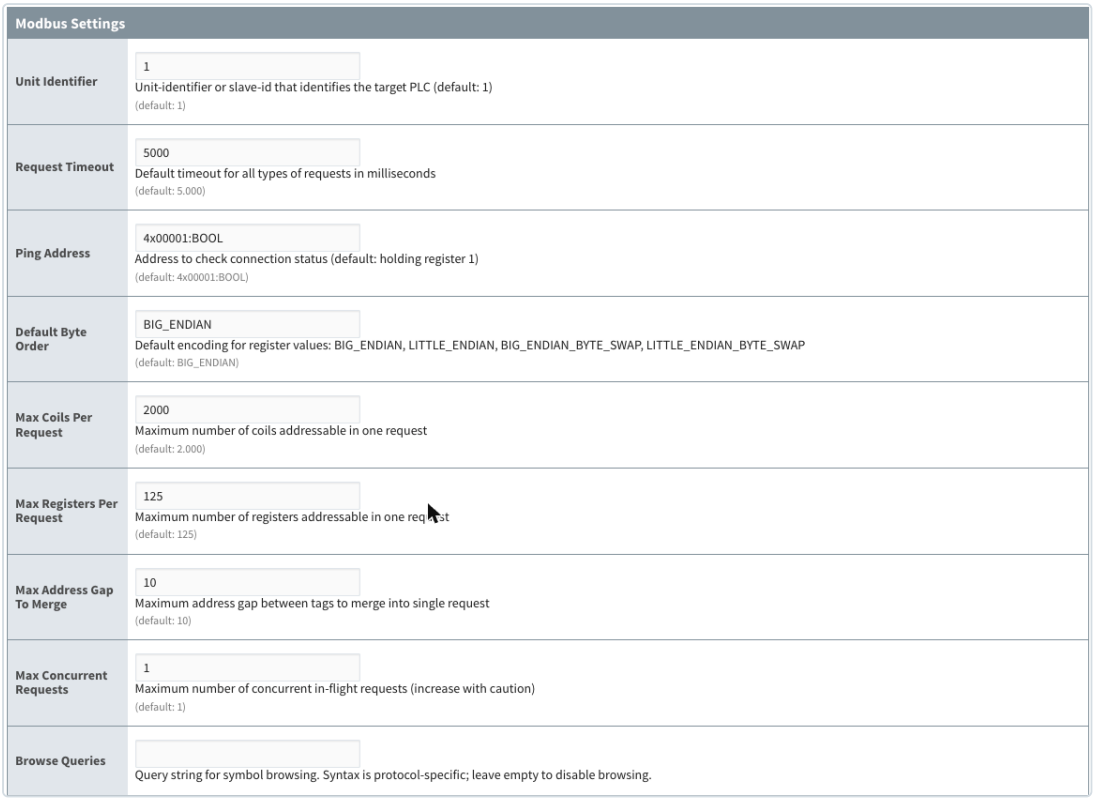
This is the name the connection is then referenced under in other parts of Ignition.

Transport Settings



Setting	Description	Default
Host	Hostname or IP address of the Modbus device	
Port	TCP port number	`502`

Modbus Settings



Setting	Description	Default
Unit Identifier	Unit-identifier / slave-id of the target device	`1`
Request Timeout	Milliseconds allowed for one request	`5000`
Ping Address	Address read to check that the connection is still alive	`4x00001:BOOL`
Default Byte Order	Default decoding for multi-register values	`BIG_ENDIAN`
Max Coils Per Request	Most coils addressable in one request	`2000`
Max Registers Per Request	Most registers addressable in one request	`125`
Max Address Gap To Merge	Largest gap between tags still merged into one request	`10`
Max Concurrent Requests	In-flight requests allowed at once	`1`
Browse Queries	Inert on this driver — Modbus cannot enumerate tags	

Default Byte Order

Modbus specifies the order of *registers* in a multi-register value but not the order of bytes within them, so vendors genuinely differ and there is no correct default. A wrong setting does not fail — it returns plausible nonsense, which is the failure mode to watch for: a `REAL` that reads as a huge or tiny number, or an integer whose bytes are transposed.

Four values are accepted: `BIG_ENDIAN`, `LITTLE_ENDIAN`, `BIG_ENDIAN_BYTE_SWAP` and `LITTLE_ENDIAN_BYTE_SWAP`. This is the connection-wide default; individual tags can override it (see *Per-tag options*), which is what you need for a device that is inconsistent with itself.

Request sizing and merging

Max Coils Per Request (`2000`) and **Max Registers Per Request** (`125`) are the protocol's own ceilings. Many devices accept less than the maximum; if a device answers small reads and errors on large ones, lower these before suspecting anything else.

Max Address Gap To Merge lets the driver coalesce nearby tags into one request, reading a few unwanted registers to save a round trip. Raising it trades wire bytes for fewer requests. Beware of merging *across* a gap that the device treats as illegal — some devices fault a read that spans an unmapped address, in which case lower this.

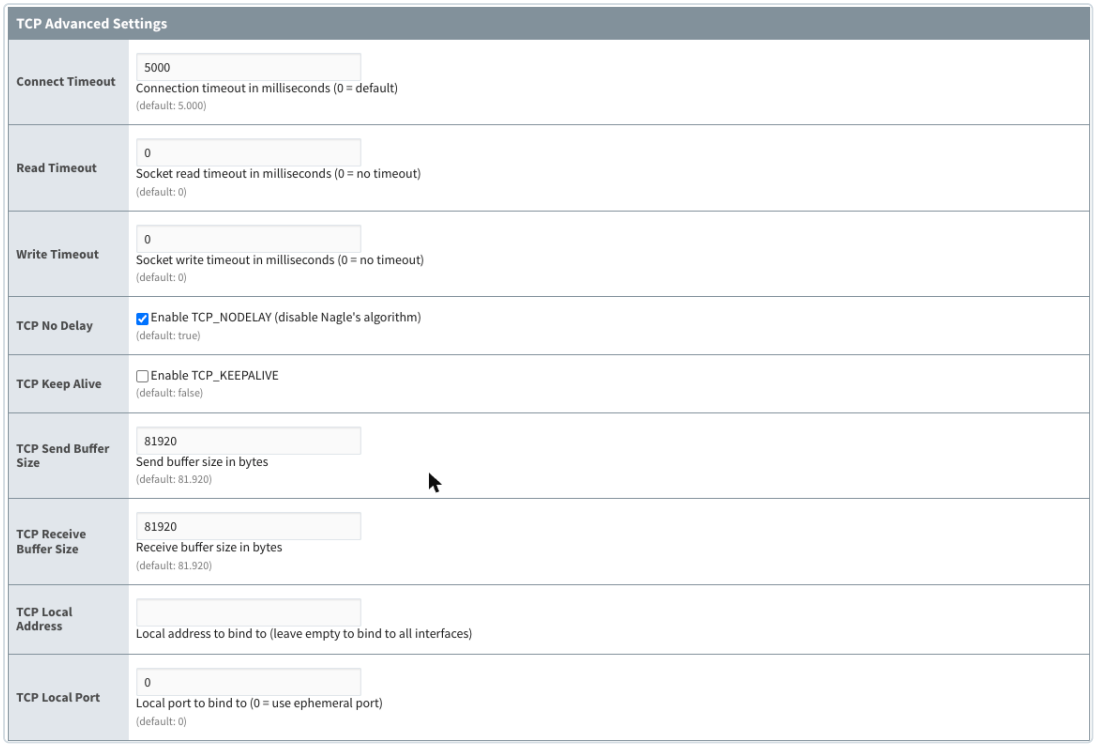
Max Concurrent Requests defaults to `1`, and the description's "increase with caution" is earned: many Modbus devices are single-threaded and answer concurrent requests with errors, dropped frames or transposed responses. Raise it only for a device documented to support it.

Ping Address

The address read periodically to confirm the connection is alive. The default, `4x00001:BOOL`, is holding register 1 — which exists on most devices but not all. If a device faults on the ping while ordinary tags read correctly, point this at a register you know is mapped.

TCP Advanced Settings

Socket-level tuning. The defaults are appropriate for nearly every device; change them for a specific diagnosed problem rather than as routine optimisation.

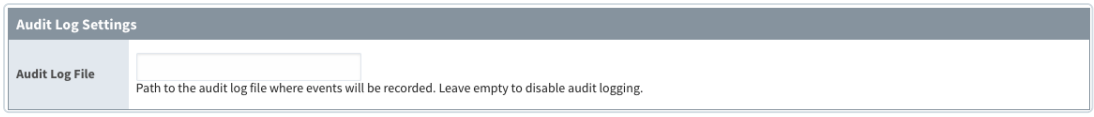


Setting	Description	Default
Connect Timeout	Connection timeout in milliseconds (`0` = default)	`5000`
Read Timeout	Socket read timeout in milliseconds (`0` = no timeout)	`0`
Write Timeout	Socket write timeout in milliseconds (`0` = no timeout)	`0`
TCP No Delay	Enable `TCP_NODELAY`, disabling Nagle's algorithm	`true`
TCP Keep Alive	Enable `TCP_KEEPALIVE`	`false`
TCP Send Buffer Size	Send buffer size in bytes	`81920`
TCP Receive Buffer Size	Receive buffer size in bytes	`81920`
TCP Local Address	Local address to bind to; empty binds all interfaces	
TCP Local Port	Local port to bind to (`0` = ephemeral)	`0`

TCP No Delay is on by default because Modbus exchanges are small request/response pairs, exactly the pattern Nagle's algorithm delays.

TCP Local Address is occasionally needed on a multi-homed gateway, to make the connection leave by a particular interface.

Audit Log Settings



Setting	Description	Default
Audit Log File	Path to a file protocol audit events are written to; empty disables	

The audit log records protocol-level events and grows quickly, so enable it only while investigating a problem.

Tag Addressing

```
<area-prefix>:<starting-register>[:<datatype>][[<selection>]]
[{<option>: <value>[, ...]}]
```

Area prefixes are `holding-register`, `input-register`, `coil` and `discrete-input`; Schneider's `%MW0` / `%M0` form is also accepted.

Address	Meaning
`holding-register:40001:INT`	One signed 16-bit int
`holding-register:40001:DINT`	Signed 32-bit int spanning registers 40001–40002
`holding-register:40001:REAL`	IEEE-754 float spanning 40001–40002
`holding-register:40001:STRING(20)`	A 20-character string starting at 40001
`holding-register:40001[0..9]:DINT`	Ten 32-bit ints, registers 40001–40020
`holding-register:40001[9]:DINT`	**One** DINT, the tenth
`holding-register:40001[0..19]:CHAR`	Twenty individual characters, as a list
`coil:1`	A single coil
`coil:1[0..7]:BOOL`	Eight coils as a BOOL array

> **A bare `[N]` is one element, not a count of N**, and the selection goes **before** the `:TYPE` suffix. Both changed together in ToddySoft Connect 1.2 so that an address written for an earlier release **fails to parse** rather than quietly returning one register where it used to return ten. Each parse error names its replacement.

`STRING` and `WSTRING` must carry a length, because nothing on the wire announces one: `STRING(20)[3]` is three 20-character strings, while `CHAR[20]` is twenty separate characters. A `STRING` without a length is rejected when the address is parsed.

Per-tag options

A trailing `{…}` overrides connection settings for one tag.

Option	Values	Meaning
`unit-id`	0–255	Address this tag to a specific unit / slave
`byte-order`	`BIG_ENDIAN`, `LITTLE_ENDIAN`, `BIG_ENDIAN_BYTE_SWAP`, `LITTLE_ENDIAN_BYTE_SWAP`	Byte order for this tag, overriding the default

```
holding-register:100:INT{unit-id: 5}
holding-register:100[0..9]:DINT{unit-id: 5, byte-order:
LITTLE_ENDIAN}
```

`unit-id` is how you reach several devices behind one connection — a serial multi-drop line behind a Modbus gateway, typically. Tags addressed to different units are never merged into one request, so a device with many units costs at least one request per unit.

Tag Groups

By default, all tags are collected in the default tag group, which means all selected tags are fetched at the default interval of once a second.

If different intervals are desired, create additional tag groups, configure the desired intervals there, and assign tags to the corresponding tag groups.

Modbus has no change notification, so subscriptions are served by polling and the tag group interval *is* the poll rate. Tags that sit close together in the address space and are read at the same interval merge into one request — so grouping by *both* address locality and rate is what keeps the request count down.

Licence diagnosis

When a device reports a licence problem, the module explains it on the gateway itself. Open **Status > Other > ToddySoft Licence (Modbus)**. A device with a licence problem says so in its status and points there.

The diagnosis shows:

- **which gateway** this is, and its role if it is part of a redundant pair;
- **Ignition's licence** for this module on this gateway: activated, trial running (with its end) or trial expired;
- **what ToddySoft licence data arrived** with Ignition's licence: how many parts, whether they are complete, and a checksum that identifies the licence without revealing it;
- the **driver's own verdict** and the **cause**, in words.

Cause	Meaning	What to do
`IGNITION_UNLICENSED`	Ignition applies no activated licence to this module on this gateway (trial running or expired). ToddySoft licence data only arrives with an activated licence.	Activate a licence key that covers this module on this gateway. In a redundant pair every node needs its own activation.
`NO_DATA_FOR_DRIVER`	The activated key carries no ToddySoft licence for this driver.	Send the report to ToddySoft.
`DATA_INCOMPLETE`	The licence data arrived incomplete or damaged.	Send the report to ToddySoft; the licence has to be delivered again.
`DATA_REJECTED`	The driver refused the delivered licence; the reason is shown.	Send the report to ToddySoft.
`RUNNING_ON_BUNDLED_TRIAL`	A licence was delivered but not accepted, and the driver runs on its built-in trial until that ends.	Send the report to ToddySoft.
`NOT_CHECKED`	Licence data arrived, but the driver gave no verdict - for instance because it could not be loaded.	Send the report to ToddySoft.

Cause	Meaning	What to do
`VALID`	Licensed; tier and expiry are shown.	Nothing.
`NOT_YET_KNOWN`	The first seconds after start-up, before Ignition reports a licence state.	Wait a moment.

Sending a report to support. The diagnosis comes as a plain-text report, the one thing to send with any licence question. The page shows it ready to select and copy. The same text is served at `http://<gateway>:<port>/data/com.toddysoft.connect.modbus/licence-report` to a logged-in user. It contains no licence data, no keys, no credentials and no customer details. In a redundant pair, send the report from both nodes.

The gateway log carries the same report once each time the licence situation changes.

Troubleshooting

Licensing problems after a new or renewed licence key was activated — `DATA_INCOMPLETE` with "duplicate fragment index" in the licence diagnosis:

- **Two licence keys covering this module are activated on the gateway at the same time**, typically a renewed or upgraded key activated alongside the old one. Ignition hands the module the licence data of both, in numbered parts that carry the same numbers; the module cannot tell which parts belong together, keeps one of each pair, and the licence it reassembles is unreadable. Depending on the parts it keeps, devices are refused or run on the module's bundled trial until that ends.
- Keep exactly **one** activated licence key that covers this module per gateway: deactivate the old one under **Config > System > Licensing**, then restart the module (or the gateway). In a redundant pair, check every node.

The module will not load, or the log shows `ClassNotFoundException` / `NoClassDefFoundError`:

- Ignition's **OPC UA** module is missing or disabled. This driver is an OPC UA device driver and cannot load without it. Install or re-enable it under **Config > System > Modules** and restart the gateway — see Installation

Device shows "Faulted":

- Verify the device is reachable and TCP port 502 (or your override) is open
- Check the **Unit Identifier** — a device behind a gateway rarely answers on the default `1`
- Check the **Ping Address** exists on this device; the default is holding register 1
- Review Gateway logs: **Status > Diagnostics > Logs**

Values are plausible but wrong — floats absurd, integers transposed:

- Almost always **Default Byte Order**. Try the other three values; override per tag with `{byte-order: …}` for a device that is inconsistent with itself

Small reads work, larger ones fail:

- Lower **Max Registers Per Request** / **Max Coils Per Request**. The protocol ceilings are not what every device accepts
- Or lower **Max Address Gap To Merge**: a merged read may be spanning an unmapped address the device refuses

Intermittent errors, dropped or mismatched responses:

- Set **Max Concurrent Requests** back to `1`. Many devices are single-threaded

Nothing to browse in the Designer:

- Expected. Modbus has no tag enumeration; write addresses by hand from the device's Modbus map

An address that used to work no longer parses:

- The array grammar changed: the selection goes before the type, and `[N]` is one element rather than a count. The error message names the replacement

Docker related issues

The Modbus protocol carries no identity for this end of the connection, so a container's internal address is never announced to the device and nothing has to be reconciled. A gateway in a bridged container connects normally, provided the device's port is routable.

VPN related issues

A remote device reached over a VPN needs nothing special from this driver: the connection is an ordinary outbound TCP session. Note that latency matters more here than on most drivers, because Modbus is strictly request/response — with **Max Concurrent Requests** at `1`, every tag read costs a full round trip.

Version history

Unreleased — 1.3.0

- First release of this manual.
- **New:** licence diagnosis — why a device is or is not licensed, shown on the gateway, with a report to send to support. See Licence diagnosis.

- **New:** troubleshooting for two licence keys covering this module activated on one gateway at the same time. See Troubleshooting.